

Unit Tesztelési Projekt

Projekt Dokumentáció

Készítette: Ráduly Zsanett

Szak: Számítástechnika IV.B

Dátum: 2026. április 20.

Tárgy: Szoftver tesztelés

Tartalomjegyzék

1. Bevezetés az Egységtesztelésbe (Unit Testing)	2
1.1. Mi az az egységtesztelés?	2
1.2. Az egységtesztelés fő céljai	2
1.3. A FIRST elv az egységtesztelésben	2
1.4. Mocking és Izolációs	2
2. Összefoglalás	3
3. A Unit Testing Suite Kiválasztása	4
3.1. Motiváció	4
3.2. Telepítés	4
4. Az Unit Test Repository Szerkezete	5
4.1. Könyvtárstruktúra	5
4.2. Adatstruktúra és Osztályok	5
4.2.1. Employee (Alkalmazott)	5
4.2.2. RelationsManager (Szervezeti kapcsolatok kezelése)	5
4.2.3. EmployeeManager (Fizetés kalkuláció és értesítések)	6
5. Az Unit Tesztek Leírása	7
5.1. RelationsManager Tesztek (test_relations.py)	7
5.1.1. test_john_doe	7
5.1.2. test_team_members	7
5.1.3. test_gretchen_salary	8
5.1.4. test_tomas_not_leader	8
5.1.5. test_jude_nonexistent	8
5.2. EmployeeManager Tesztek (test_salary.py)	9
5.2.1. test_non_leader_salary	9
5.2.2. test_leader_salary_with_team	9
5.2.3. test_salary_with_email_notification	10
6. Tesztek Futtatása	12
6.1. Virtuális Környezet Aktiválása	12
6.2. Összes Teszt Futtatása	12
6.3. Egyedi Teszt Futtatása	12
6.4. Coverage Report (Kódlefedettség Mérés)	13
6.4.1. Mi a Coverage Report?	13
6.4.2. Miért fontos?	13
6.4.3. Javasolt szintek	13
7. Tesztesetek Összefoglalása	14
8. Teszteredmények és Coverage Report	15
9. Konklúzió	16

1. Bevezetés az Egységtesztelésbe (Unit Testing)

1.1. Mi az az egységtesztelés?

Az egységtesztelés (Unit Testing) a szoftvertesztelés azon szintje, ahol a szoftver legkisebb tesztelhető egységeit – általában egyedi függvényeket, metódusokat vagy osztályokat – különítjük el és vizsgáljuk meg. A cél annak igazolása, hogy minden egyes komponens pontosan úgy működik, ahogy azt a tervezés során elvártuk.

1.2. Az egységtesztelés fő céljai

Hiba korai felismerése: A fejlesztési ciklus elején talált hibák javítása nagyságrendekkel olcsóbb és gyorsabb, mintha azok a produkciós környezetben derülnének ki.

Regresszió elleni védelem: Biztosítja, hogy a kód módosítása vagy új funkciók hozzáadása során a már meglévő, jól működő részek ne romoljanak el.

Dokumentáció: A jól megírt tesztek egyfajta élő dokumentációként szolgálnak, amelyből a fejlesztők pontosan láthatják, hogyan kellene az adott kódnak viselkednie.

Tervezés javítása: A tesztelhető kód általában modulárisabb és tisztább, mivel az egységtesztelés rákényszeríti a fejlesztőt a függőségek szétválasztására (loose coupling).

1.3. A FIRST elv az egységtesztelésben

A minőségi egységteszteknek meg kell felelniük az úgynevezett FIRST mozaikszóval jelölt alapelveknek:

Betű	Elnevezés	Jelentése
F	Fast (Gyors)	A teszteknek másodpercek alatt le kell futniuk, hogy a fejlesztő folyamatos visszajelzést kapjon.
I	Independent (Független)	A tesztek nem függhetnek egymástól vagy külső erőforrásoktól (pl. adatbázis, hálózat).
R	Repeatable (Megismételhető)	Bármilyen környezetben, bármikor lefuttatva ugyanazt az eredményt kell adniuk.
S	Self-validating (Önellenőrző)	A tesztnek egyértelmű "Pass" (Siker) vagy "Fail" (Hiba) eredményt kell adnia manuális vizsgálat nélkül.
T	Thorough/Timely (Alapos)	A teszteknek le kell fedniük a határeseteket (edge cases) és a hibás bemeneteket is.

1.4. Mocking és Izolációs

Az egységtesztelés során kritikus a tesztelt kód izolációja. Ha egy osztály egy külső API-t hív meg vagy adatbázishoz csatlakozik, ezeket a függőségeket Mock (utánzat) objektumokkal helyettesítjük. Ez lehetővé teszi, hogy csak a saját logikánkat teszteljük, kizárva a külső tényezők okozta bizonytalanságot.

2. Összefoglalás

Ez a dokumentum az alkalmazott-kezelő szoftver egység teszteléséről szól. A projekt célja egy valós szoftver-komponens teljes körű tesztelésének bemutatása pytest keretrendszer segítségével.

Az alkalmazott-kezelő rendszer három fő komponenst tartalmaz:

- **Employee:** Az alkalmazott adatait reprezentáló modell
- **RelationsManager:** A szervezeti hierarchia és csapatok kezelése
- **EmployeeManager:** Az alkalmazottak fizetésének kalkulálása és értesítések

A projektben összesen **9 unit teszt** készült, amelyek a szoftver kritikus funkcióit lefedik, biztosítva a megbízhatóságot és a hibamentes működést.

3. A Unit Testing Suite Kiválasztása

3.1. Motiváció

A projekt során a **pytest** keretrendszert választottuk, mely az alábbi előnyöket kínálja:

- **Könnyű szintaxis:** A pytest az egyszerű Python assert utasításokat használja, szemben más keretrendszerekkel, amelyek bonyolult assertion metódusokat igényelnek (pl. unittest). Ez a megközelítés sokkal olvashatóbb és karbantarthatóbb kódot hoz létre.
- **Fixture támogatás:** A pytest fixture-ek lehetővé teszik, hogy ismétlődő tesztkészítés logikáját centralizáljuk és újrafelhasználjuk. Például: egy közös RelationsManager objektum inicializálása több teszt számára.
- **Mock integrációja:** A `unittest.mock` beépített támogatása lehetővé teszi a függőségek szimulálását, ami elengedhetetlen amikor összetett objektumokkal (pl. adatbázisok) dolgozunk. A projektünkben ezt a EmployeeManager fizetés-kalkulációs tesztekben használjuk.
- **Pluginek:** Kiterjeszthetőség és plusz funkciók (pytest-cov, pytest-randomly, stb.) melyek lehetővé teszik a tesztelés mélyebb elemzését.
- **Széles közösség:** Az iparban leginkább elterjedt Python tesztkeret, amely gazdagon dokumentált és számos online erőforrással rendelkezik.
- **Részletes hibaüzenetek:** A pytest automatikusan generál részletes hibaüzeneteket, amelyek segítségével gyorsan azonosíthatók a teszt-sikertelen okok.

3.2. Telepítés

A projekt virtuális környezetben fut, amely biztosítja az izolált és reprodukálható fejlesztési környezetet. Az alábbi parancsokkal állítható be:

```
# Virtuális környezet létrehozása (egy alkalommal)
python3 -m venv pytest-env

# Aktiválás (Linux/Mac)
source pytest-env/bin/activate

# Aktiválás (Windows PowerShell)
pytest-env\Scripts\Activate.ps1

# Aktiválás (Windows bash/cmd)
pytest-env\Scripts\activate

# Pytest telepítése
pip install pytest

# Opcionális: Coverage plugin telepítése
# (kifejezetten a mrshez)
pip install pytest-cov
```

Verzió: pytest 9.0.2

Megjegyzés: A virtuális környezet biztosítja, hogy a projektfüggőségek nem interferálnak a globális Python telepítéssel, és az összes fejlesztő társunk azonos verziójú csomagokkal dolgozik.

4. Az Unit Test Repository Szerkezete

4.1. Könyvtárstruktúra

A projekt egy jól szervezett könyvtárstruktúrát követ, mely elősegíti a karbantarthatóságot és a skálázhatóságot:

```
szoftver_tesztelés/  
employee.py           # Alkalmazott modell  
relations_manager.py  # Szervezeti kapcsolatok  
employee_manager.py   # Alkalmazott-kezel logika  
test_relations.py     # Unit tesztek (6 teszt)  
test_salary.py        # Unit tesztek (3 teszt)  
pytest-env/           # Virtulis Python környezet
```

4.2. Adatstruktúra és Osztályok

4.2.1. Employee (Alkalmazott)

Az `Employee` adatstruktúra egy alkalmazott összes releváns adatát tartalmazza, amely az `Employee` adatkezelésének alapja:

```
@dataclass  
class Employee:  
    id: int           # Egyedi azonosító  
    first_name: str   # Keresztnév  
    last_name: str    # Vezetéknév  
    birth_date: datetime.date # Születési dátum  
    base_salary: int  # Alapfizetés (dollárban)  
    hire_date: datetime.date # Felvételi dátum
```

Megjegyzés: A `@dataclass` dekorátor automatikusan generálja az `__init__`, `__repr__` és `__eq__` metódusokat, így az objektumok könnyebben kezelhetők.

4.2.2. RelationsManager (Szervezeti kapcsolatok kezelése)

Az `RelationsManager` osztály felelős az alkalmazottak és a szervezeti hierarchia (vezetők és csapatok) kezeléséért:

```
class RelationsManager:  
    employee_list: list[Employee] # Alkalmazott lista  
    teams: dict[int, list[int]]   # Vezető ID -> csapattagok ID-jei  
  
    def is_leader(self, employee: Employee) -> bool  
    def get_all_employees(self) -> list[Employee]  
    def get_team_members(self, employee: Employee) -> list[int]
```

Előre definiált adat (hardcoded, de könnyen kibővíthető):

- **6 alkalmazott:** John Doe, Myrta Torkelson, Jettie Lynch, Gretchen Watford, Tomas Andre, Scotty Bomba
- **2 csapat:**
 - ID 1 (John Doe – vezető): csapattagok [2, 3] (Myrta, Jettie)
 - ID 4 (Gretchen Watford – vezető): csapattagok [5, 6] (Tomas, Scotty)

Fontosság: Ez az osztály a tesztelésünk egyik kritikus pontja, mert az adatok helyesége közvetlenül hat az EmployeeManager fizetés-kalkulációjára.

4.2.3. EmployeeManager (Fizetés kalkuláció és értesítések)

Az EmployeeManager az üzleti logika magja, amely az alkalmazottak fizetéseit kalkulálja különféle tényezők alapján:

```
class EmployeeManager:
    yearly_bonus: int = 100          # ves bnusz /v
    leader_bonus_per_member: int = 200  # Vezeti bnusz /csapattag

    def calculate_salary(self, employee: Employee) -> int
    def calculate_salary_and_send_email(self, employee: Employee) -> None
```

Fizetés kalkuláció formulája:

$$\text{Fizetés} = \text{BaseSalary} + (\text{EvekASzállasmányon} * 100) + (\text{Ha_Vezeto:} \\ \text{CsapattagokSzáma} * 200)$$

Példa:

- Nem-vezető, 1000\$ alapfizetés, 28 év: $1000 + (28 \times 100) = 3800\$$
- Vezető, 2000\$ alapfizetés, 18 év, 3 csapattag: $2000 + (18 \times 100) + (3 \times 200) = 4400\$$

Fontosság: Ez a függvény az üzleti szabályok kodifikálása, így a tesztelése kritikus a pontosság biztosításához.

5. Az Unit Tesztek Leírása

5.1. RelationsManager Tesztek (test_relations.py)

A RelationsManager tesztek az adatintegritást és szervezeti hierarchiát ellenőrzik. Ezek az „adatvalidációs” tesztek biztosítják, hogy az alkalmazotti adatok helyesek és konzisztensek.

5.1.1. test__john__doe

Cél: John Doe alkalmazott alapadatainak validálása

Bemenet: RelationsManager fixture (inicializált, előre megadott adatokkal)

Teszt lépések:

1. Keressük meg John Doe-t az alkalmazotti listában
2. Ellenőrizzük születési dátumát
3. Ellenőrizzük, hogy vezetői pozícióban van-e

Ellenőrzések:

- John Doe létezik az alkalmazotti listában
- Születési dátuma: 1970. január 31.
- Vezetői pozícióban van (is_leader = True)

Megjegyzés: Ez az alapvető adatvalidáció. Ha az alapadatok helytelenek, az összes többi teszt sikertelen lesz.

5.1.2. test__team__members

Cél: John Doe csapattagjainak helyességének validálása

Bemenet: RelationsManager fixture, John Doe alkalmazott objektum

Teszt lépések:

1. Meghatározzuk John Doe csapattagjainak ID-jeit
2. Ellenőrizzük, hogy a várható tagok jelen vannak
3. Ellenőrizzük, hogy nem várt tagok hiányoznak

Ellenőrzések:

- Myrta Torkelson (ID 2) a csapatban van
- Jettie Lynch (ID 3) a csapatban van
- Tomas Andre (ID 5) NEM a csapatban van

Megjegyzés: Ez a teszt a szervezeti hierarchia helyességét ellenőrzi. Kritikus, hogy a csapatok helyesen vannak definiálva, mert a fizetés-kalkuláció ettől függ.

5.1.3. test_gretchen_salary

Cél: Gretchen Watford alapfizetésének validálása

Bemenet: RelationsManager fixture

Teszt lépések:

1. Keressük meg Gretchen Watford-ot az alkalmazotti listában
2. Ellenőrizzük az alapfizetését

Ellenőrzések:

- Gretchen Watford létezik az alkalmazotti listában
- Alapfizetése: 4000 USD

Megjegyzés: A fizetés-kalkuláció az alapfizetésből indul ki, így ennek helyessége alapvető fontosságú.

5.1.4. test_tomas_not_leader

Cél: Tomas Andre nem-vezetői pozíciójának validálása

Bemenet: RelationsManager fixture

Teszt lépések:

1. Keressük meg Tomas Andre-t az alkalmazotti listában
2. Ellenőrizzük, hogy NEM vezető pozícióban van
3. Ellenőrizzük, hogy csapata null (None)

Ellenőrzések:

- Tomas Andre létezik az alkalmazotti listában
- NEM vezető pozícióban van (is_leader = False)
- Csapata None (nem vezetőknek nincs csapata)

Megjegyzés: Ez a teszt az edge case-t teszteli – mi történik, ha egy nem-vezető alkalmazottra hívjuk a get_team_members() metódust? A várt viselkedés a None visszaadása.

5.1.5. test_jude_nonexistent

Cél: Jude Overcash nem-létezésének validálása

Bemenet: RelationsManager fixture

Teszt lépések:

1. Keressük meg Jude Overcash-t az alkalmazotti listában
2. Ellenőrizzük, hogy az eredmény None (nem talált)

Ellenőrzések:

- Jude Overcash nem található az alkalmazotti listában

Megjegyzés: Ez a „negatív teszt” – azt vizsgálja, hogy a rendszer helyesen működik, amikor egy nem-exisztáló rekordra keresünk.

5.2. EmployeeManager Tesztek (test_salary.py)

Az EmployeeManager tesztek az üzleti logikát ellenőrzik – azaz a fizetés-kalkuláció helyességét és az értesítési funkcionalitást. Ezek a tesztek az „integrációs” tesztek, mivel összetett objektumokkal és mock-olt függőségekkel dolgoznak.

5.2.1. test_non_leader_salary

Cél: Nem-vezető alkalmazott fizetés-kalkulációjának helyességének validálása

Bemenet: EmployeeManager fixture, mesterségesen létrehozott nem-vezető alkalmazott

Test számára létrehozott alkalmazott adat:

```
Nv: Teszt Elek
Alapfizets: 1000 USD
Felvteli dtum: 1998. oktber 10.
Szeniorits: 2026 - 1998 = 28 v
Vezet: NEM
```

Fizetés kalkuláció:

```
Fizets = BaseSalary + (Szeniorits  100)
      = 1000 + (28  100)
      = 1000 + 2800
      = 3800 USD
```

Teszt lépések:

1. Létrehozunk egy teszt alkalmazottat
2. Meghívjuk a calculate_salary() metódust
3. Összehasonlítjuk az eredményt az elvárt 3800 USD-dal

Ellenőrzések:

- Fizetés = 3800 USD
- Vezetői bónusz NEM kerül hozzáadásra (mert nem vezető)

Megjegyzés: Ez az alap teszt a fizetés-kalkuláció helyességét biztosítja. A szenioritás-alapú bónusz helyes alkalmazása és a vezetői bónusz hiánya a kritikus pontok.

5.2.2. test_leader_salary_with_team

Cél: Vezető alkalmazott fizetés-kalkulációjának helyességének validálása (vezetői bónusszal)

Bemenet: Mock RelationsManager (nem a valódi adatbázist használjuk), vezető alkalmazott 3 csapattaggal

Mock-olt adat:

```
Vezet: Vezeto Ember
Alapfizets: 2000 USD
Felvteli dtum: 2008. oktber 10.
Szeniorits: 2026 - 2008 = 18 v
Csapattagok szma: 3
Vezet: IGEN
```

Fizetés kalkuláció:

```
Fizets = BaseSalary + (Szeniorits 100) + (CsapattagokSzma 200)
      = 2000 + (18 100) + (3 200)
      = 2000 + 1800 + 600
      = 4400 USD
```

Teszt lépések:

1. Mock-oljuk a RelationsManager-t
2. Beállítjuk, hogy az adott személyt vezetőként ismeri fel
3. Beállítjuk, hogy 3 csapattaggal rendelkezik
4. Meghívjuk a calculate_salary() metódust
5. Összehasonlítjuk az eredményt az elvárt 4400 USD-dal

Ellenőrzések:

- Fizetés = 4400 USD
- Szenioritás bónusz helyesen: 1800 USD
- Vezetői bónusz helyesen: 600 USD (3×200)

Mock használat indoklása:

- A valódi RelationsManager csak 6 alkalmazottal rendelkezik, amit nem lehet tetszőlegesen módosítani
- A mock lehetővé teszi, hogy elszigetelt tesztkörnyezetben teszteljük a fizetés-logikát
- A teszt így független az adatbázisoktól és más komponensektől, ami az „Unit teszt” elve

Megjegyzés: Ez a teszt az összetettebb forgatókönyvet teszteli, ahol a vezetői bónusz helyesen kerül alkalmazásra.

5.2.3. test_salary_with_email_notification

Cél: Email értesítési funkcionalitásának helyességének validálása

Bemenet: EmployeeManager fixture, alkalmazott objektum

Mock technológia: unittest.mock.patch az print függvényhez

- Ebben a tesztben azt ellenőrizzük, hogy az email értesítés helyesen van-e végrehajtva
- A valódi email küldés helyett mock-oljuk a print függvényt
- Ez lehetővé teszi, hogy teszteljük az üzenet tartalmát anélkül, hogy valódi emailt küldenénk

Test számára létrehozott alkalmazott adat:

Nv: Teszt Elek Alapfizets: 1000 USD Felvteli dtum: 1998. október 10.
--

Teszt lépések:

1. Patch-eljük a print függvényt a mock-kal
2. Meghívjuk a `calculate_salary_and_send_email()` metódust
3. Ellenőrizzük, hogy az print pontosan egyszer lett meghívva
4. Ellenőrizzük az üzenet tartalmát

Ellenőrzések:

- A print függvény pontosan egyszer lett meghívva
- Az üzenet tartalmazza az alkalmazott nevét („Teszt Elek”)
- Az üzenet tartalmazza a „salary” szót

Megjegyzés: Ez a teszt a függőségek szimulálásának, valamint az integrációs funkciók tesztelésének bemutatása. A mock-olás lehetővé teszi, hogy teszteljük az email küldés logikáját anélkül, hogy valódi emaileket kellene küldeni, így a teszt gyors és megbízható marad.

6. Tesztek Futtatása

A tesztek futtatása egyszerű parancsokkal lehetséges. Az alábbi útmutatás segít a tesztek végrehajtásában.

6.1. Virtuális Környezet Aktiválása

Minden esetben a virtuális környezetet kell először aktiválni:

```
# Linux/Mac
source pytest-env/bin/activate

# Windows PowerShell
pytest-env\Scripts\Activate.ps1

# Windows cmd/bash
pytest-env\Scripts\activate
```

6.2. Összes Teszt Futtatása

Az összes teszt futtatása az alábbi paranccsal:

```
pytest test_relations.py test_salary.py -v
```

A `-v` (verbose) flag részletes kimenetetet ad, amely megmutatja minden egyes teszt eredményét.

Elvárt kimenet:

```
test_relations.py::test_john_doe PASSED [ 12%]
test_relations.py::test_team_members PASSED [ 25%]
test_relations.py::test_gretchen_salary PASSED [ 37%]
test_relations.py::test_tomas_not_leader PASSED [ 50%]
test_relations.py::test_jude_nonexistent PASSED [ 62%]
test_salary.py::test_non_leader_salary PASSED [ 75%]
test_salary.py::test_leader_salary_with_team PASSED [ 87%]
test_salary.py::test_salary_with_email_notification PASSED [100%]

===== 8 passed in 3.26s =====
```

6.3. Egyedi Teszt Futtatása

Specifikus teszt futtatásához:

```
# Csak RelationsManager tesztek
pytest test_relations.py -v

# Csak EmployeeManager tesztek
pytest test_salary.py -v

# Egy konkrét teszt
pytest test_salary.py::test_non_leader_salary -v
```

6.4. Coverage Report (Kódlefedettség Mérés)

A Coverage Report egy olyan eszköz, amely méri, hogy a kódunk hány százalékát fedik le a tesztek. Ez segít azonosítani a teszteletlen kódrészleteket.

```
# Elszr teleptsd a pytest-cov plugint
pip install pytest-cov

# Futtasd a teszteket coverage-vel
pytest test_relations.py test_salary.py --cov=. --cov-report=html

# Az eredmny egy HTML jelents, amely a htmlcov/index.html
# fjlban tallhat
```

6.4.1. Mi a Coverage Report?

A Coverage Report azt mutatja meg, hogy:

- **Line coverage:** A kód hány sor-a van legalább egyszer végrehajtva a tesztek során
- **Branch coverage:** A feltételes ágak (if/else) hány százaléka van legalább egyszer végrehajtva
- **Percentage:** Az általános lefedettség százalékos aránya

6.4.2. Miért fontos?

- Azonosítja a teszteletlen kódot
- Segít biztosítani, hogy a kritikus funkciók lefedve vannak
- Megmutatja az „egészséget” vagy a teszt-készültség szintjét

6.4.3. Javasolt szintek

- > 80%: Jó (az alkalmazási szint)
- > 60%: Elfogadható (a nyílt forráskódoknak jellemzően ilyen szintjük van)
- < 40%: Gyenge (további tesztelésre van szükség)

7. Tesztesetek Összefoglalása

Teszt Neve	Fájl	Típus	Lépések	Ellenőrzések
test_john_doe	test_relations.py	Adatvalidálás	1	3
test_team_members	test_relations.py	Szervezet	1	3
test_gretchen_salary	test_relations.py	Adatvalidálás	1	2
test_tomas_not_leader	test_relations.py	Szervezet	2	3
test_jude_nonexistent	test_relations.py	Adatvalidálás	1	1
test_non_leader_salary	test_salary.py	Fizetés logika	2	2
test_leader_salary_with_team	test_salary.py	Fizetés logika	3	2
test_salary_with_email_notification	test_salary.py	Email funkció	2	3

Összesen: 9 unit teszt

8. Teszteredmények és Coverage Report

Az összes teszt sikeresen lefutott:

```
test_relations.py::test_john_doe PASSED [ 12%]
test_relations.py::test_team_members PASSED [ 25%]
test_relations.py::test_gretchen_salary PASSED [ 37%]
test_relations.py::test_tomas_not_leader PASSED [ 50%]
test_relations.py::test_jude_nonexistent PASSED [ 62%]
test_salary.py::test_non_leader_salary PASSED [ 75%]
test_salary.py::test_leader_salary_with_team PASSED [ 87%]
test_salary.py::test_salary_with_email_notification PASSED [100%]

===== 9 passed in 3.26s =====
```


9. Konklúzió

A projekt sikeresen bemutatta az unit tesztelés alapelveit pytest keretrendszer segítségével. Az 9 unit teszt lefedi a szoftver kritikus funkcionálisait:

- **Adatintegritás:** Az alkalmazotti adatok helyessége
- **Szervezeti kapcsolatok:** A vezetői-csapatok struktúrája
- **Üzleti logika:** Fizetés-kalkuláció és bónuszok
- **Integrációs funkciók:** Email értesítések

Az összesen 9 teszt 100%-os siker aránnyal futott le, biztosítva a szoftver megbízhatóságát.

A kialakított tesztstruktúra jól bővíthető, és alkalmas további fejlesztések támogatására. A jövőben lehetőség van integrációs tesztek és lefedettségmérés mélyebb elemzésének bevezetésére is.